

Code obfuscation is a method of securing source code. OWASP Recommendation: It protects the source code by making it difficult for both humans and artificial intelligence to analyze or understand it. The code's functionality remains the same after obfuscation. The code still works, but it looks more complex and harder to understand than before the obfuscation.

Before Control Flow:

```
1 public int CompareTo (Object o) {
2     int n=occurrences-
3     ((WordOccurrence)o).occurrences;
4     if (n=0) {
5         n=String.Compare
6         (word, ((WordOccurrence)o).word);
7     }
8     return (n);
9 }
10
11
12
13
```

After Control Flow:

```
1 private virtual int _a(Object A+0) {
2     int local0;
3     int local1;
4     local 10=this.a-(c) A_0.a;
5     if (local10 !=0) goto i0;
6     while (true) {
7         return local1;
8     }
9     i1:local10=
10 System.String.Compare(this.b, (c)
11 A_0.b);
12     goto i0;
13 }
```

Concealment is nothing.

Code obfuscation is sometimes confused with other techniques. Differentiating these techniques from code obfuscation will help clarify its meaning.

Data obfuscation versus code obfuscation

Data obscurity is the attempt to hide sensitive data items. Code obscurity is a narrower, more specialized attempt to conceal the functionality of code, such as obscuring code flow (see below). Here are some examples of data obscurity used in cybersecurity.

Concealment (or anonymization, token encryption): This involves replacing data with random characters or codes to hide sensitive information.

Information editing: Information editing is the process of removing sensitive information from a file or program.

Encoding: This process converts information into a secret code that conceals its true meaning.

Compare code obfuscation techniques with other techniques.

Code obfuscation is often mentioned alongside other security techniques and is sometimes confused with them.

Anti-tampering: This is a general term for any measure that makes it more difficult for an attacker to modify source code, such as checking the integrity of the source code during execution to ensure it has not been modified.

Copyright marking (or forensic copyright marking): This involves embedding a unique identifier into a program for traceability purposes.

Control during operation: This includes root and jailbreak detection, anti-debugging, and anti-tampering system techniques.

The problems that blurring techniques are designed to solve.

Obfuscation of source code reduces its readability and comprehension. By making the code more complex, obfuscation slows down attacks by professional hackers and acts as a psychological barrier to deter attacks from amateur hackers and "script kiddies" (hackers who only know how to use simple attack tools).

The problem with reverse engineering

The main goal of obfuscating source code is to make reverse engineering easier. reverse engineering- Obfuscation of the code makes it more difficult to understand malicious code and program logic. Reverse engineering attacks targeting applications can aim to steal intellectual property, make malicious changes, or leak data. The first step in carrying out such an attack is to thoroughly understand the application involved and then plan the cyberattack. Obfuscation of the source code makes understanding the code more difficult, making reverse engineering efforts a more time-consuming, frustrating, and painful process.

Preventing common attacks

Obfuscation of source code has the potential security benefit of making vulnerabilities harder for attackers to find and exploit. Attacks often rely on small details of the program's internal structure. Deliberate obfuscation creates diversity and variability in deployed applications. This can make it more difficult for malicious actors to understand the source code or build exploits that work effectively against multiple targets on a large scale.

Prevent unauthorized software modification.

Obfuscation of source code doesn't actively prevent modification. However, it offers the indirect benefit of making the software more difficult to access and modify

illegally. If the source code is difficult to understand, it means it will be harder to modify.

Code obfuscation techniques are different from anti-tampering mechanisms, such as checksums. Checksums are a process of verifying the integrity of code to protect it from modification. However, these two types of techniques are often combined to preserve the intended functionality of the application code.

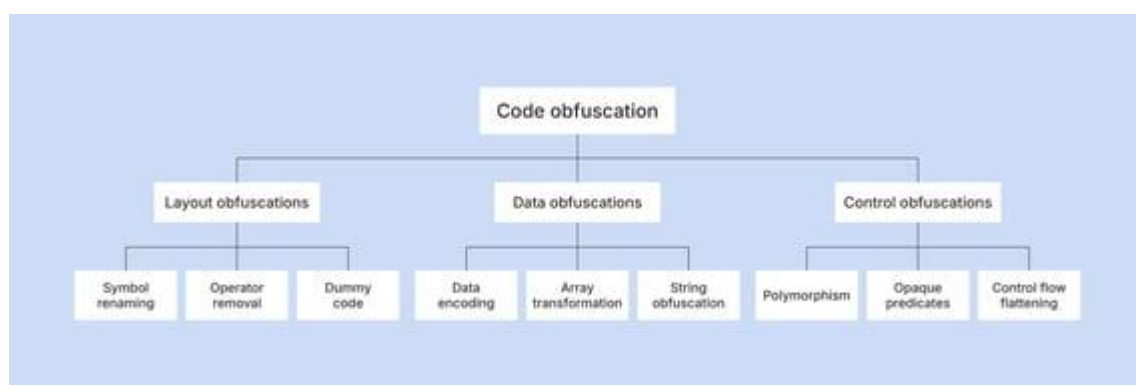
Types of techniques for obfuscating the main code.

To better understand how code obfuscation works, it's helpful to consider how the individual techniques are grouped together. Each code obfuscation technique will transform one of these aspects of the code.

Control flow obfuscation techniques: These are techniques used to obscure the flow of code, altering and concealing the program's workflow, making it difficult for attackers to analyze. Control flow obfuscation is one of the most common and widely used methods of source code obfuscation.

Data obfuscation: This method works by obfuscating and obfuscating the program's data structure. This can be done by targeting arrays, obfuscating variables, and class transformations.

Obfuscation of layout or vocabulary: This technique targets the program's layout structure by renaming identifiers and removing source code formatting. These are one-way techniques—once the information is deleted, there is no way to restore the original formatting.



Limitations and potential problems of the source code obfuscation process.

Encryption, or source code obfuscation, is widely used by high-security applications across many industries, including banking, gaming, and streaming. It provides a cost-effective security enhancement solution that can be quickly and easily

implemented with the right tools. When applied, encryption is very difficult to bypass and protects against a wide range of attacks.

However, despite these advantages, the effectiveness of code obfuscation still has limitations. Many of these weaknesses arise when obfuscation is used alone, abused, or employed inexperiencedly. The good news is that these drawbacks are significantly reduced if obfuscation is combined with other security techniques, and the least impactful obfuscation techniques are used intelligently. High-quality obfuscation tools should aim for this goal.

Performance issues

Obfuscation of source code is often praised because it creates a practical balance between security effectiveness and performance. However, excessive obfuscation can negatively impact application performance, leading to a poor user experience. It can even result in the application being rejected on app stores.

One way to address this overload problem is to prioritize obfuscating critical components that could be exploited over obfuscating unimportant code. Decide which code is necessary (e.g., sensitive code) and which is not (e.g., performance-critical code, uncritical code).

Protection issues

Obfuscation of source code offers only limited defense against execution-time attacks, such as dynamic metering. It only addresses the behavior of the code and does not specifically protect the stored data (e.g., API keys, user login credentials, tokens, local databases). And it's not a perfect method against attackers with the motivation, skill, and time. It acts as a speed barrier, not a wall. That's why it's often combined with real-time control measures in the context of protecting mobile applications.

The role of source code obfuscation in application security.

According to a Gartner report on *Avoiding Mobile App Security Pitfalls* There is a five-stage roadmap for building a secure application: threat modeling, security development, testing, security enhancement, and anti-tampering. Source code obfuscation plays a central role in the final stage of these two key areas of application security and in-app protection. This involves security features that can be integrated into mobile applications and combined with them to prevent threats.

Concealing information is a measure to enhance application security.

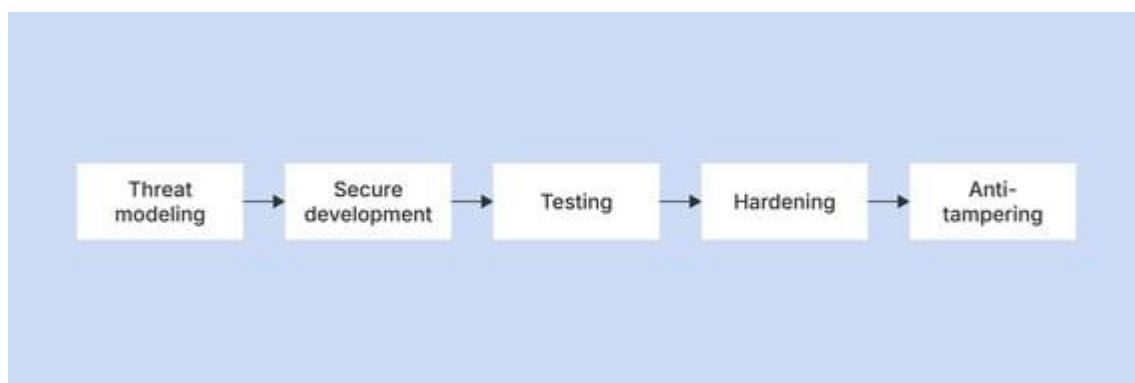
Enhanced application security is the process of strengthening an application's security by reducing the size of its vulnerable attack surface. This is achieved by modifying the application's source code, binary code, or bytecode to make it more resistant to interference and denial-of-service (RE) attacks.

Some techniques enhance application security. It has the ability to predict, monitor, and detect attacks. Other techniques—such as source code obfuscation and data encryption—help prevent and stop them by increasing the difficulty for attackers to perform static analysis or carry out attacks.

Concealing information as a way to protect the application.

Unlike enhancing app security, which is primarily a preventative measure, application shielding can provide comprehensive protection for mobile applications against attacks and threats in real time. Along with code obfuscation, it includes data obfuscation techniques (such as white box code), as well as runtime security technologies to protect the entire application ecosystem at multiple levels.

Runtime application protection mechanisms control and monitor the application's operation on the device to ensure it remains clean and secure. This software security mechanism is called... Protect the application at runtime. (RASP). Integrating static obfuscation encryption with dynamic RASP checks creates a more comprehensive security posture.



The role of code obfuscation in application development

There are two ways to consider where code obfuscation occurs during application development.

Code blurring in SDLC

The Software Development Life Cycle (SDLC) includes the fundamental stages in building a software application: analysis, planning, design, construction, testing, deployment, and maintenance. Debugging the source code, along with other security enhancement techniques, is crucial for the later stages of this cycle.

The process of obfuscation of the source code takes place *at the end* of the compilation phase ensures that it does not affect the code development process. However, obfuscation of the source code must be performed *before* software and performance testing. This is to ensure that the application's functionality continues to work normally after obfuscation and that the obfuscation process does not cause any errors.

During this critical build phase, source code obfuscation can be performed before, during, or after compilation (see below). Later, during the maintenance phase, the obfuscation can be reapplied to new builds and updated to maintain security. This is where finding the right balance between security and performance becomes crucial.

Deobfuscate the code and compile it.

In the typical compilation process of a compiled language, there are three places where code obfuscation can occur. These are the three levels of code obfuscation. It is important to consider the advantages of each option.

Source code level

Code obfuscation tools modify source code files before they are compiled. The downside of obfuscation here is:

- The integration process can be quite complex.
- This could cause usability issues.

Intermediate coding level

Intermediate-level code obfuscation tools and most application security tools work well at this stage. The downside to code obfuscation here is:

- This may slow down the compilation process.
- This requires integration into the toolset.
- It needs access to the source code to obfuscate it.

Binary code/machine code

Post-compilation code obfuscation means it doesn't need to happen during the development phase of the application lifecycle. Instead, it can occur after the product

has been compiled. This is the best level of code obfuscation. There are many advantages to using post-compilation and post-build code obfuscation:

It is easier to use, integrate, and maintain.

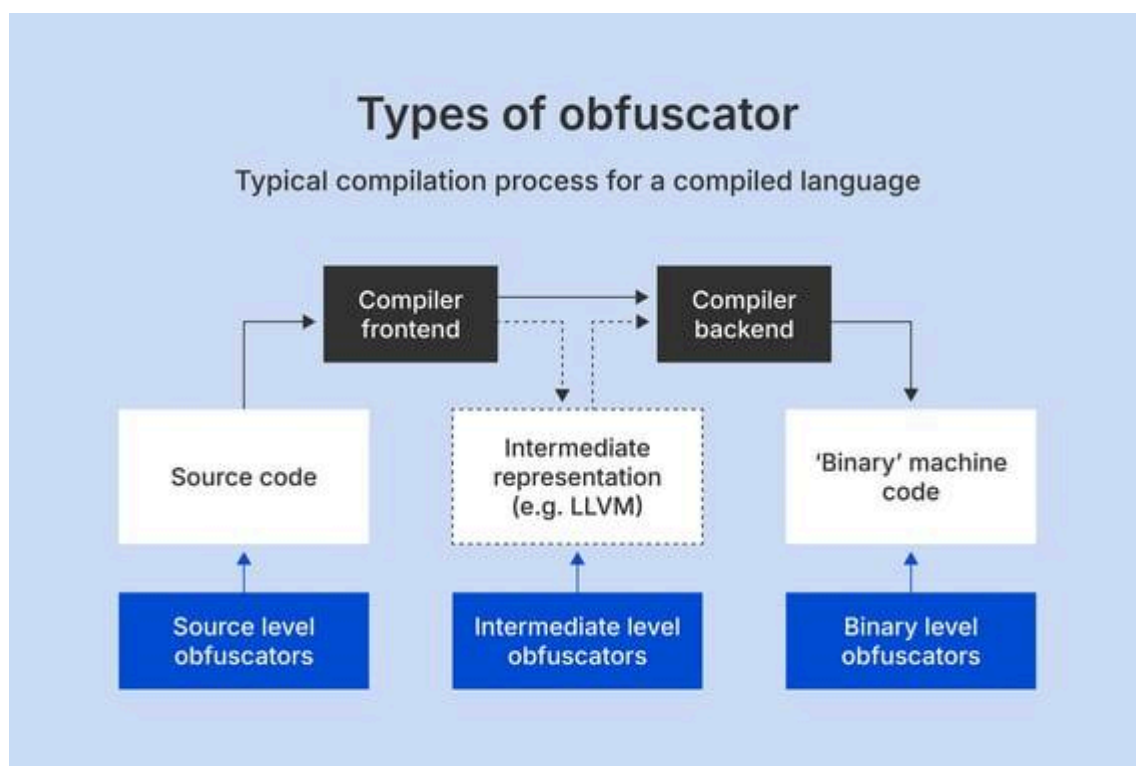
It is not bound by source code or compiler.

It does not affect integrated development environments (IDEs).

Integration with the toolkit is not necessary.

It is not dependent on any development tool, therefore it does not need to integrate with any specific development tool.

This means that security teams are not dependent on development teams/tools and can apply security policies independently of developers.



Classification is the optimal method for obfuscating source code.

There are many recommendations for best practices regarding source code obfuscation. For example, source code obfuscation should be regularly updated with the latest tools, tests, and methods. Attention should be paid to the performance of the obfuscated source code in terms of speed and size. And using variety and randomness is very smart so that each application build has a unique binary file.

But the best method for concealing information lies in creating multiple layers.

Source code obfuscation is a crucial element of application security, but it's not sufficient to function independently—nor is it supported by a single security measure. Each individual obfuscation technique typically offers relatively little benefit. Using multiple types, techniques, and layers of obfuscation alongside other security measures is the best way to establish a robust protective shield.

Multi-layered concealment technique

The combined use of multiple obfuscation techniques creates complex layers of code, making reverse engineering (RE) as difficult and time-consuming as possible. A multi-layered defense system works by combining various obfuscation techniques multiple times to maximize efficiency. The more techniques used, the more effective the defense system becomes.*intelligently*The more secure the code becomes, the better protected it is.

Below are some code obfuscation techniques commonly used in combination across multiple layers:

Renaming symbols and functions: This operation changes the names of variables and methods to meaningless identifiers.

String obfuscation: This technique hides the contents of a code string (a string of characters in programming).

Inserting pseudocode or junk code: This operation adds code that has no function and is only for auxiliary purposes to make the code harder to read.

Operator Removal: This operation removes or replaces arithmetic operators in a program (such as +, -, *, /, =, etc.) with equivalent operators that are functional but harder to read.

Fuzzy predicates: This introduces conditional statements into the source code, confusing static analysis tools, as the results are known during execution.

The control flow cannot be simplified: This creates complexity in the control flow graph that standard decompilers find difficult to simplify or optimize.

Polymorphic obfuscation encryption: This method generates different variations of the obfuscated code for each application build, so hackers have to create new strategies each time.

Flattening the control flow: This operation blurs the sequential structure of the program flow by flattening the code blocks.

Combine code obfuscation techniques with other security techniques.

Obfuscation of source code complements other security techniques, enhancing them rather than replacing them. This is especially important in areas where obfuscation is

not a viable solution. For example, Simply disrupting the process does not prevent attacks during execution. This could lead to API misuse or data theft, so it must be combined with other defenses to achieve maximum protection.

Here are some other security techniques commonly used in conjunction with source code obfuscation:

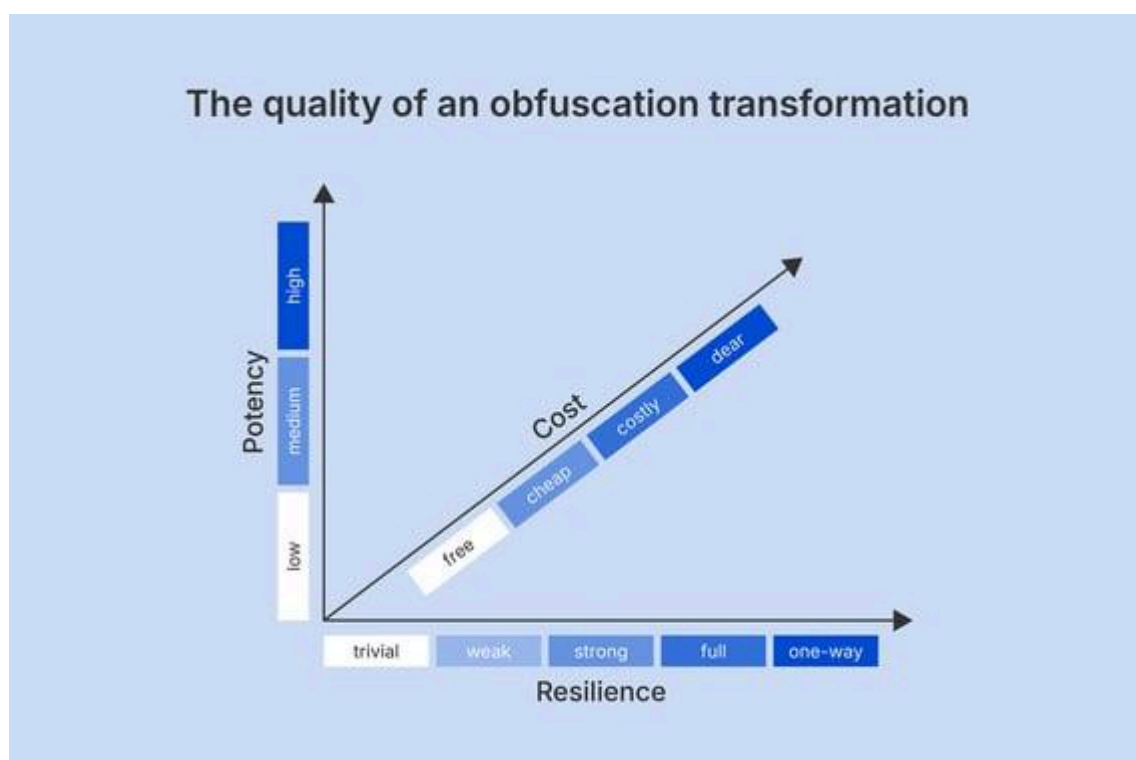
Anti-phishing solution: This is a general term for any cybersecurity measure that protects software and systems from phishing attempts. fake app .

Anti-debugging techniques: These techniques prevent malicious hackers from using tools. debugging For application analysis.

Jail and Root Detection: This function detects whether an application has been jailbroken or rooted. jailbreak or root Whether or not, these are signs that the application may have been tampered with.

Ways to measure the success of source code obfuscation.

The strength and quality of code obfuscation can be measured using a combination of technical metrics and performance testing. Technical metrics can provide some quantitative assurance for specific functionalities. However, in practice, measuring the level of code obfuscation often relies on cost-effectiveness analysis. Restoring the original code is far more expensive and difficult for an attacker than the initial obfuscation attempt.



Value

The cost of mobile device security needs to be balanced with the true cost of being unsafe. However, like any other security measure, obfuscation of source code also incurs costs and a "computational burden" on the system. The higher the level of obfuscation, the greater the cost. Humanly, this cost can be measured by increased execution time and resource consumption. Software-wise, it can lead to increased development costs, size, and performance.

Effective

The effectiveness of code obfuscation programs can be measured in several ways. Below are two of the standard measurement criteria:

Obfuscation strength: This is the degree to which the reader is confused by attempts at obfuscation, or the increased difficulty of understanding the obfuscated code. It is typically measured by comparing the original code "before" and "after" obfuscation in terms of increased complexity and obfuscation.

Resilience: Resilience against reverse engineering (RE) attacks and interference. It is a standard security metric. When applied specifically to code obfuscation, it measures how effective or robust an obfuscation technique is against automated obfuscation attempts. A highly resilient obfuscation technique remains effective even after being attacked by tools designed to bypass or reverse the obfuscation process.

These are two other criteria that also deserve attention.

Stealth: This is a measure of how seamlessly the obfuscated code integrates with the rest of the program. A high stealth rating means the obfuscation technique doesn't significantly alter the original code structure and blends well with other parts of the program, making it more difficult to detect.

Coverage: This factor considers the extent to which the program is obscured by source code obfuscation techniques. Factors such as usability and performance degradation can reduce effective coverage.

Concealing source code from the buyer's perspective.

There are several source code obfuscation products that offer similar application security capabilities. It's best to focus on the feasibility of deploying the tool and the implications for regular source code maintenance.

So the question is: Is this software package practical to use, and how sophisticated are its supporting tools? The entire software package is what matters, not just the low-level features. For example, if the tools are difficult to use, then support might be limited.

The general principles are as follows: A source code obfuscation product must be so user-friendly that it doesn't require expert expertise. It must work immediately after installation. And any necessary configurations must be clearly stated if required.

Usability

When evaluating an application security enhancement tool or product where code obfuscation is one of its primary functions, ease of use is crucial. Some tools may seem complex to configure but are difficult to use. This results in higher performance costs and potentially a thinner security coverage.

Consider which configuration options you need. Don't disable options and compromise application security with difficult-to-use or unused encryption settings.

The speed and smoothness you desire when using the tool are important factors to consider when making a purchase.

Find out what the product offers in terms of automation and installation capabilities. Options that require little to no programming code may be best suited to your needs.

Using post-compilation obfuscation tools means you don't need to be a developer to use them.

A sign of high usability is when a tool can apply basic code obfuscation to everything with minimal effort. But then, in areas where it's important, custom configurations are still possible. This flexibility in options is crucial for performance.

Efficiency

When choosing tools, try to balance the advantages in security with the limitations in performance. The benefit of obscuring source code is that it adds value to protection. But what is its impact on execution?

In reality, performance is more important than whether a product uses any specific code obfuscation techniques. Whether or not you can use those techniques depends on how easy they are to configure and how practical they are.

Choose the right analysis tool from the outset to minimize the impact of encoding configurations that obscure the source code.

If configuring a tool requires a lot of trial and error, the chances of a smooth setup are very low.

Debugging capabilities

You might launch an app on the App Store and receive a report that it's broken on certain devices. Developers discover the bug and want to fix it so they can release a new version. But once the source code is obscured, it's difficult to know what's going on.

A report might tell you that the code is failing, but it won't tell you why.

To aid in application debugging, choose software that provides mapping files to decode crash reports, which attackers would otherwise be able to access.